

Akdeniz University
CSE-214 Advanced Application Development
Final Exam

Lecturer :Dr. Alper ÖZCAN

26 May, 2025

Duration : 85 min

Name-Surname :

Student Number:

1. [25 pts] Create an **AuthService** (**auth.service.ts**) that:

1. Uses a BehaviorSubject to track if the user is logged in
2. Stores and retrieves the JWT token from localStorage.
3. Allows subscribing to the login state. **Write your code for usage in Component that can listen to changes in the user's login status**

2. [25 pts] Define a **ProductService interface** (**ProductService.java**) and its **implementation** (**ProductServiceImpl.java**). The service should include methods: **getProduct(Long id)**, **addProduct(Product product)**, and **deleteProduct(Long id)**. Use **JPA EntityManager** inside the service. You may use the following imports and do not forget to use necessary **Spring and JPA annotations**.

```
import jakarta.persistence.EntityManager;  
import jakarta.persistence.PersistenceContext;  
import org.springframework.stereotype.Service;  
import org.springframework.transaction.annotation.Transactional;
```

Hint : Assume that the Product entity class are already defined and correctly annotated.

3. [30 pts] In this question, you are required to demonstrate your understanding of object-relational mapping (ORM), entity relationships, and the use of Spring Data JPA repositories.

a. Design and implement two JPA entity classes: **Department** and **Employee**. A **department** can **contain multiple employees**, forming a **bi-directional one-to-many relationship**. Your implementation must ensure that saving **Department** instance **automatically persists** all its associated **Employee** instances. To achieve this, you are expected to use proper ORM practices, including appropriate annotations such as **@OneToMany**, **@ManyToOne**, and **CascadeType**. You must also think through and correctly apply the necessary **foreign key mapping** and ensure that the relationship is bidirectional and **lazily loaded** where appropriate.

Department Table (Main Table), Columns: id (Primary Key), name

Employee Table, Columns: id (Primary Key), name, department_id (Foreign Key)

b. Create a **Spring Data JPA repository interface** for the **Department** entity. This interface should support basic **CRUD** operations (Create, Read, Update, Delete).

Use the example code below to guide your design. Your implementation should be capable of producing similar behavior:

Example usage:

```
Department dept = new Department();  
dept.setName("IT");
```

```
Employee e1 = new Employee();  
e1.setName("Alice");  
e1.setDepartment(dept);
```

```
dept.getEmployees().add(e1);
```

```
departmentRepository.save(dept);
```

4. [20 pts] You are developing an e-commerce system that includes two entities: **Customer** and **Order**. Each **Order** is **associated with a specific Customer**. Create a **CustomerService** class that contains a method for saving a new **Customer** along with an **associated Order** in a **single transaction**. If any part of the process fails (e.g., due to a persistence error), **the entire transaction should be rolled back** to maintain **data integrity**. Apply proper **Spring and JPA annotations** where necessary to configure the entities and the service layer correctly.

Hint : Assume that the **Order** and **Customer** entity classes are already defined and correctly annotated. You are not required to rewrite these entities.

- Create **CustomerService** class and implement a method in **CustomerService** that saves **Customer** and its associated **Order** within the same transaction.
- Create **Spring Data JPA repository interface** named **CustomerRepository** to perform **CRUD** operations on the **Customer** entity.